

Transformers in Deep Learning

Lecture 3

Lecture 2 Recap:

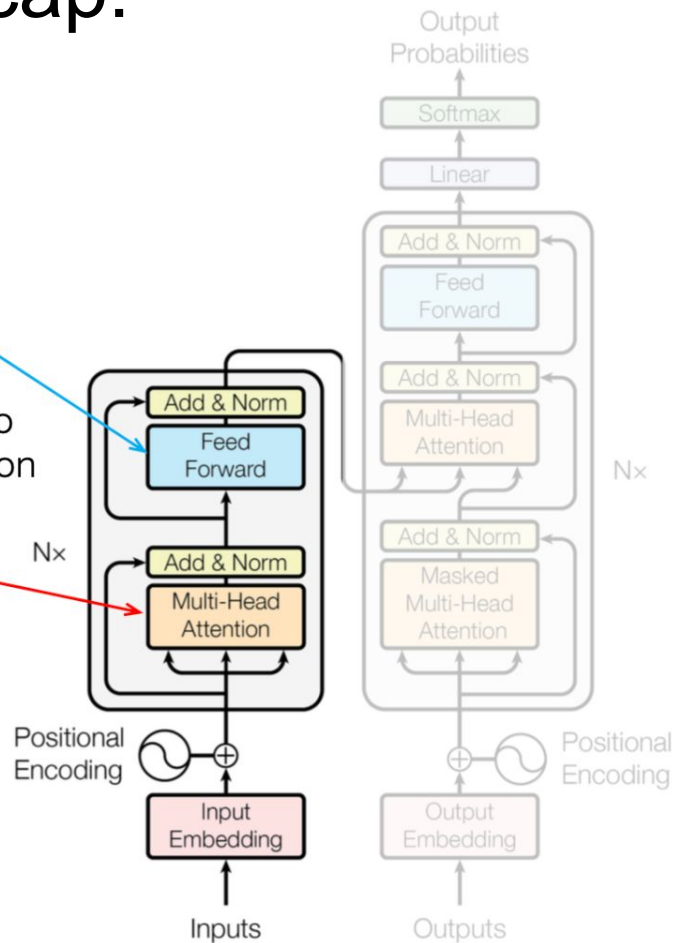
Feed-forward network:

after taking information from other tokens, take a moment to think and process this information

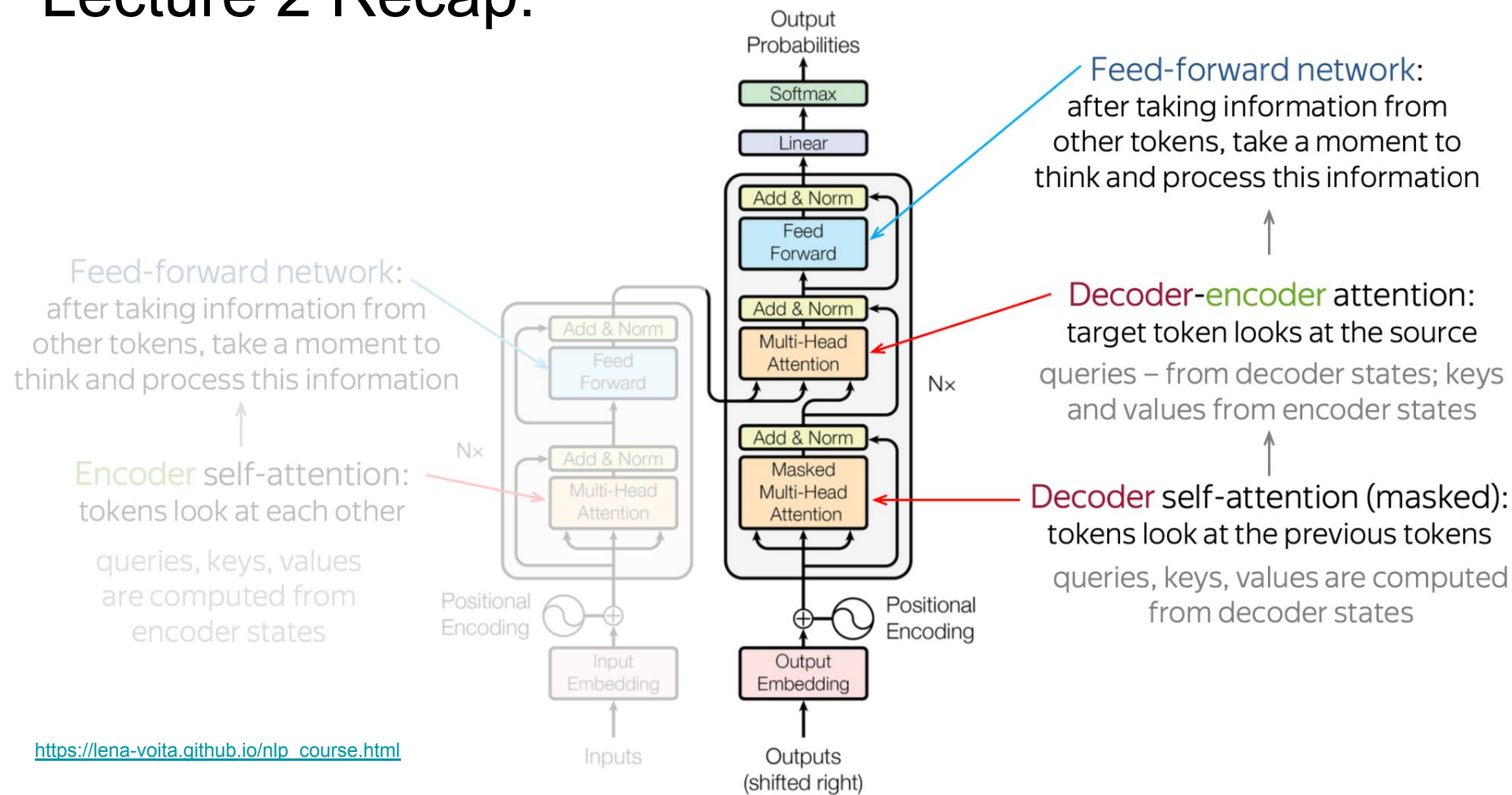
Encoder self-attention:

tokens look at each other

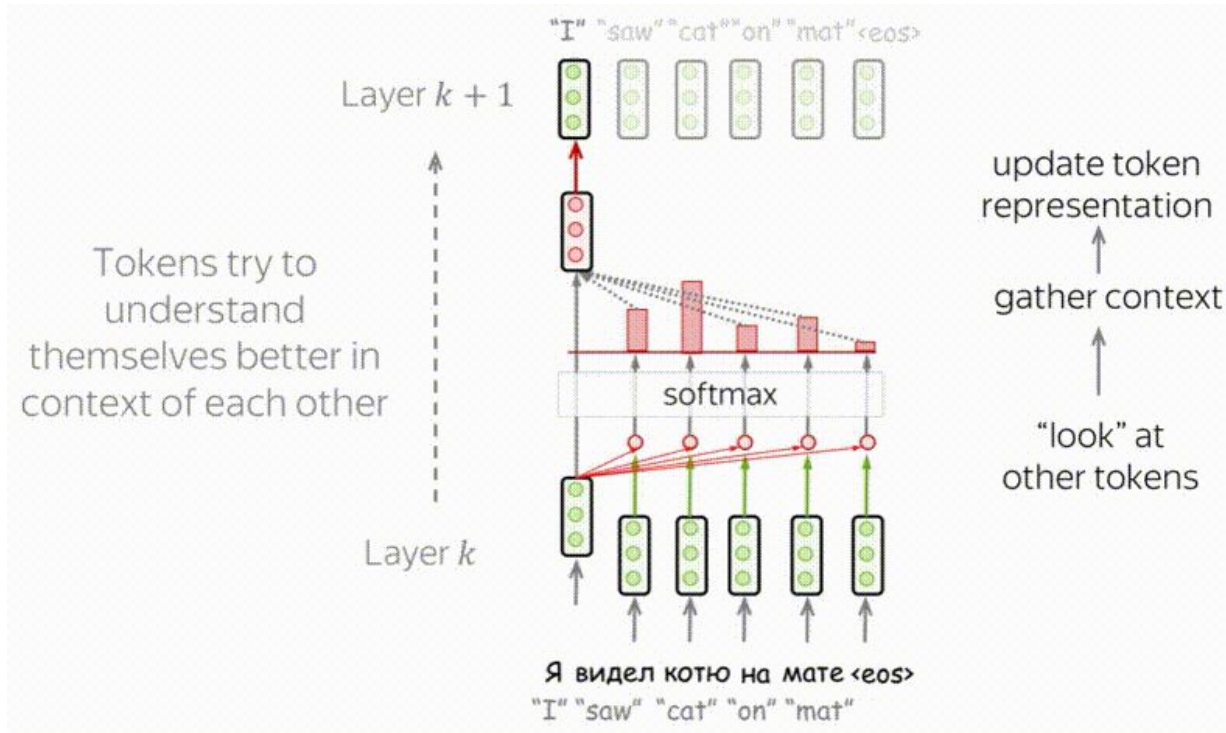
queries, keys, values are computed from encoder states



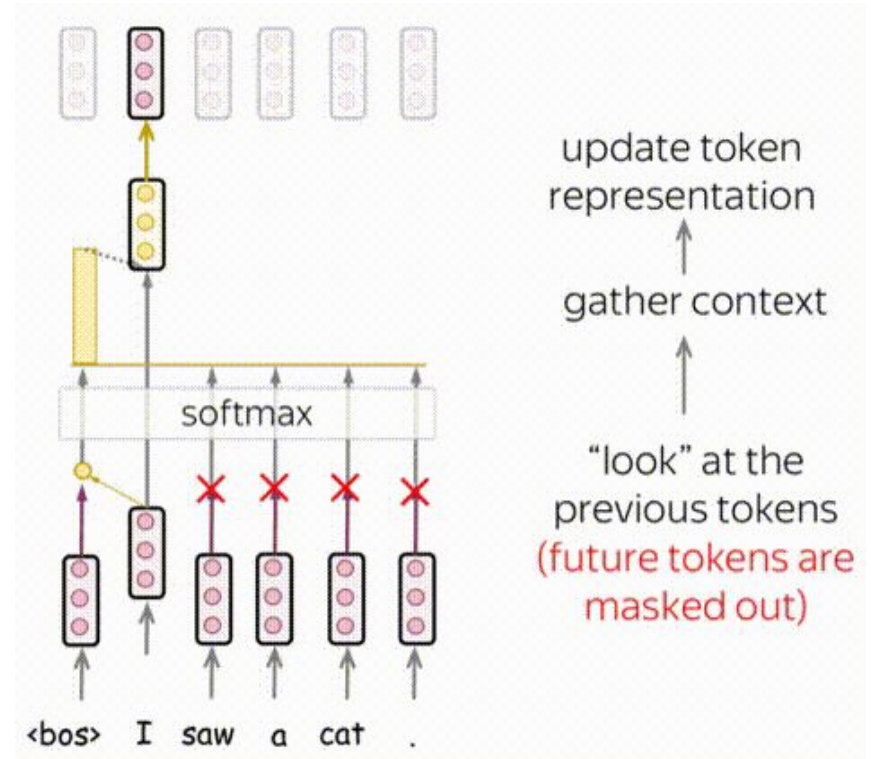
Lecture 2 Recap:



Lecture 2 Recap:



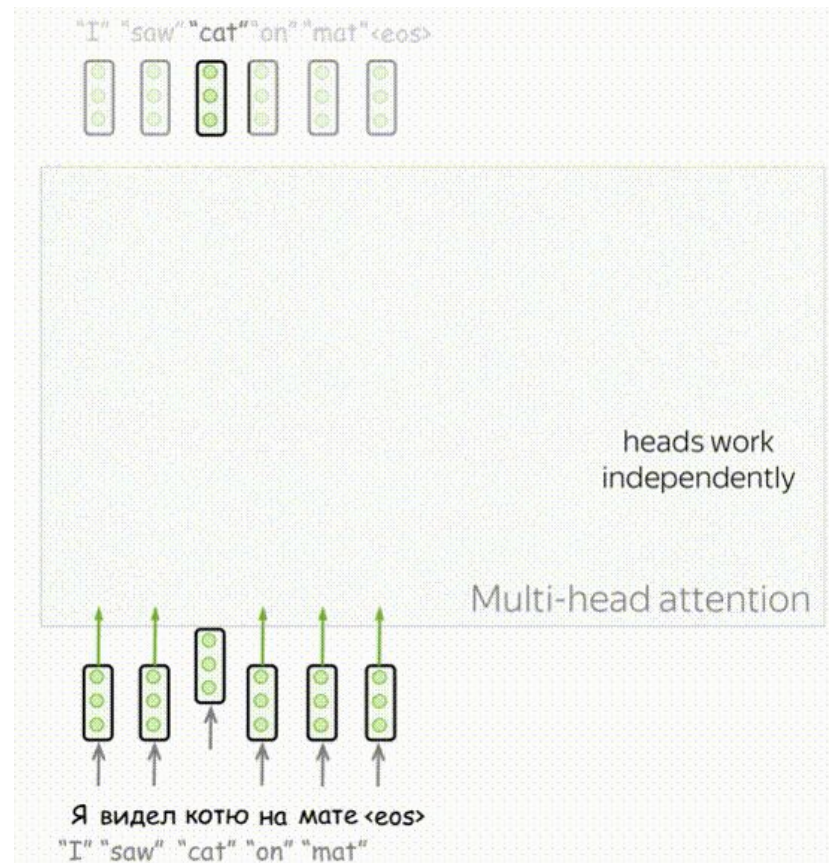
Lecture 2 Recap:



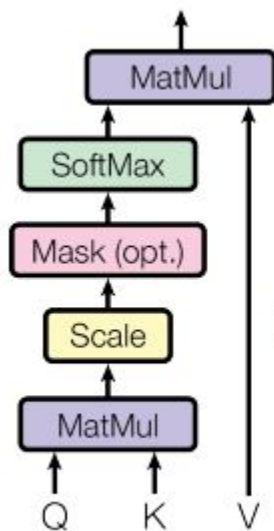
Lecture 2 Recap:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_n)W_o,$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$



Attention Recap:

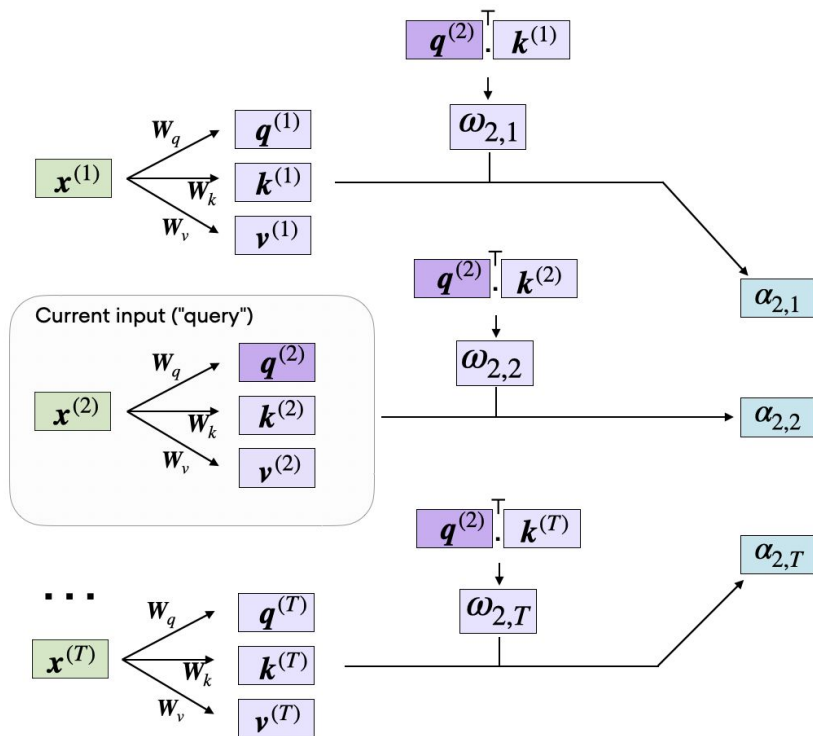


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

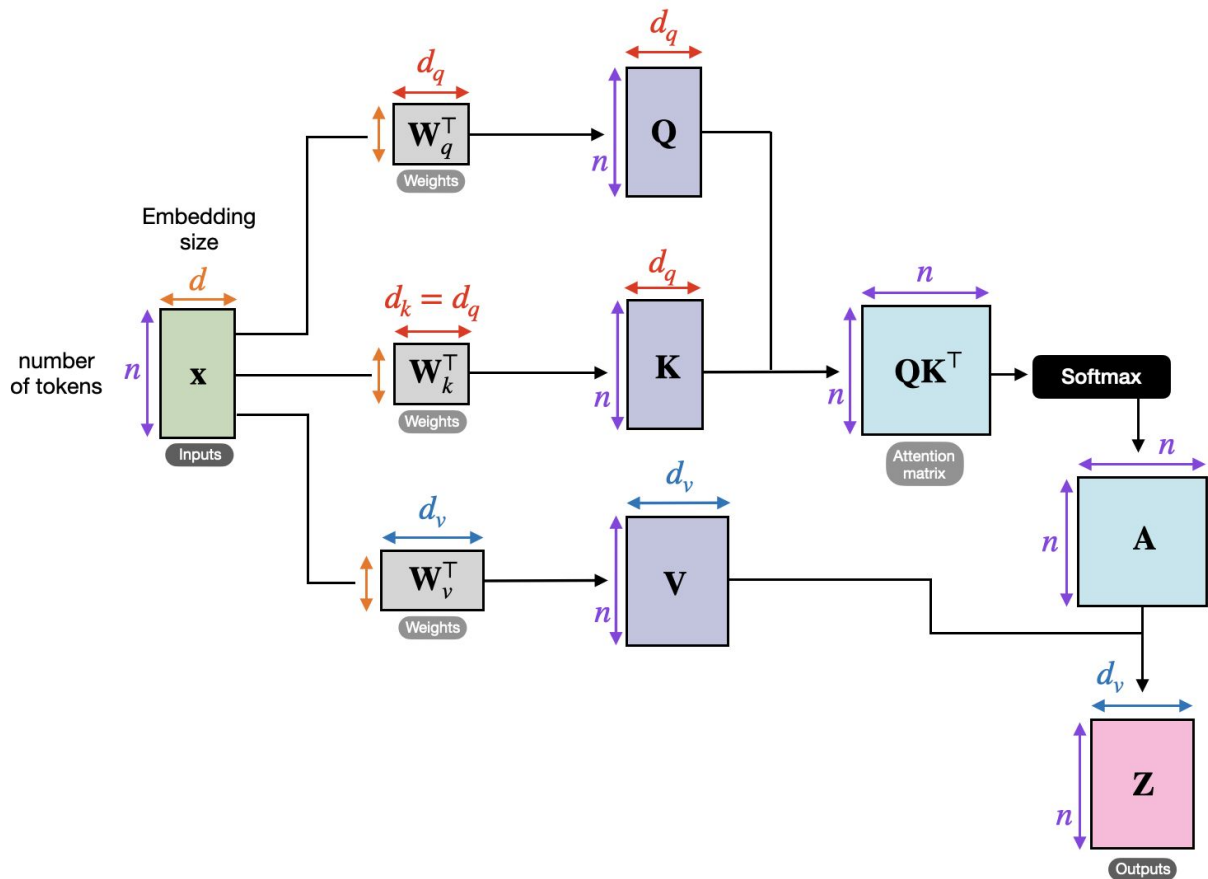
where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Attention Recap:



where $\alpha_{2,i} = \text{softmax} \left(\frac{\omega_{2,i}}{\sqrt{d_k}} \right)$

Attention Recap:



Great Idea: Transfer Learning

- Train from scratch

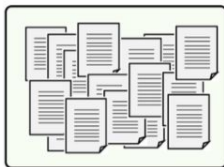
What they will know:



May be not enough
to learn relationships
between words

- Take pretrained (Word2Vec, GloVe)

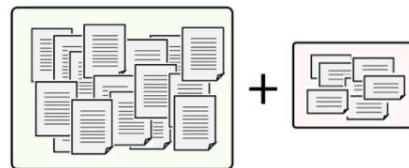
What they will know:



Know relationships between words,
but are not specific to the task

- Initialize with pretrained, then fine-tune

What they will know:



Know relationships between
words and adapted for the task

Great Idea: Transfer Learning

- Train from scratch

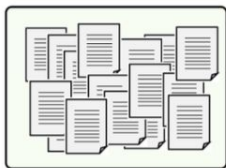
What they will know:



May be not enough
to learn relationships
between words

- Take pretrained (Word2Vec, GloVe)

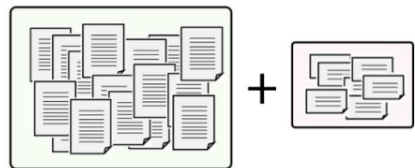
What they will know:



Know relationships between words,
but are not specific to the task

- Initialize with pretrained, then fine-tune

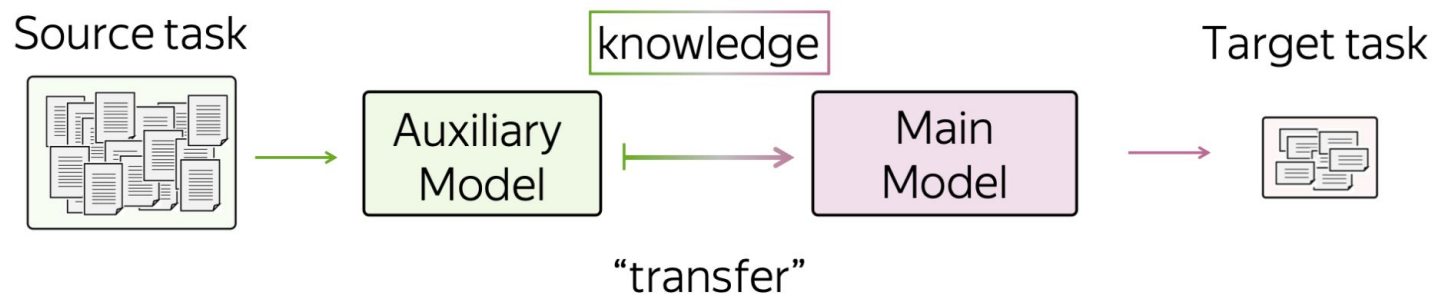
What they will know:



Know relationships between
words and adapted for the task

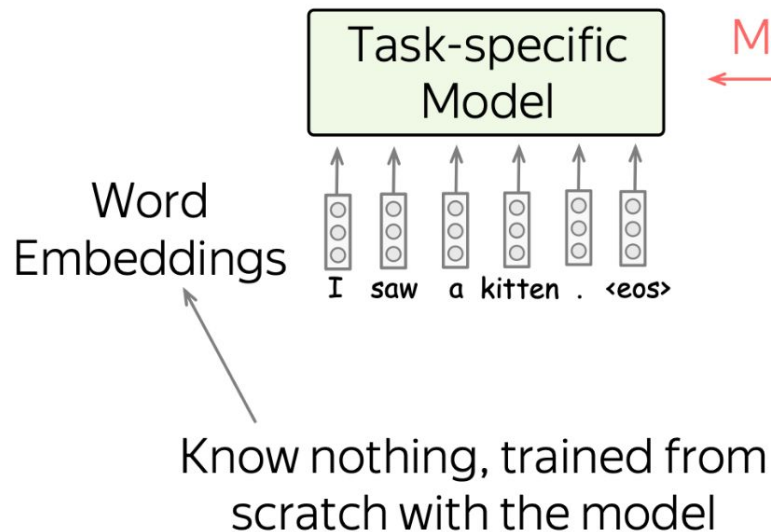
“Transfer” knowledge from a huge unlabeled
corpus to your task-specific model

Transfer Learning



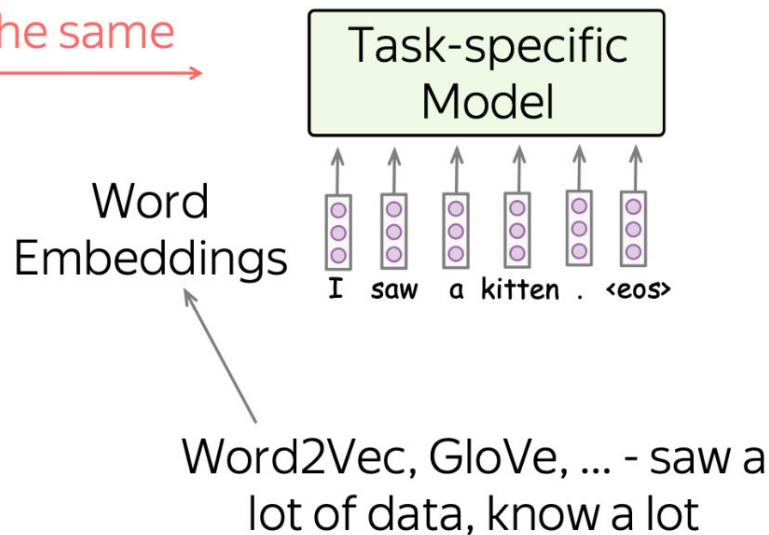
Transfer through embeddings

Before



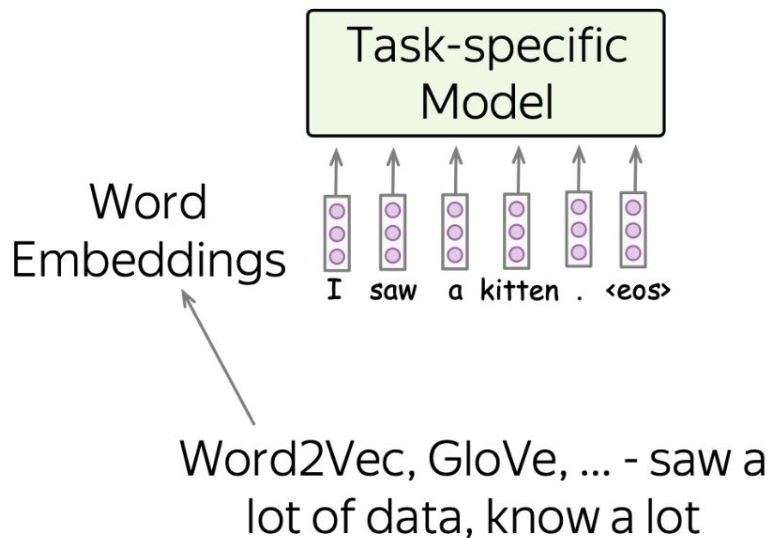
Model is the same

After

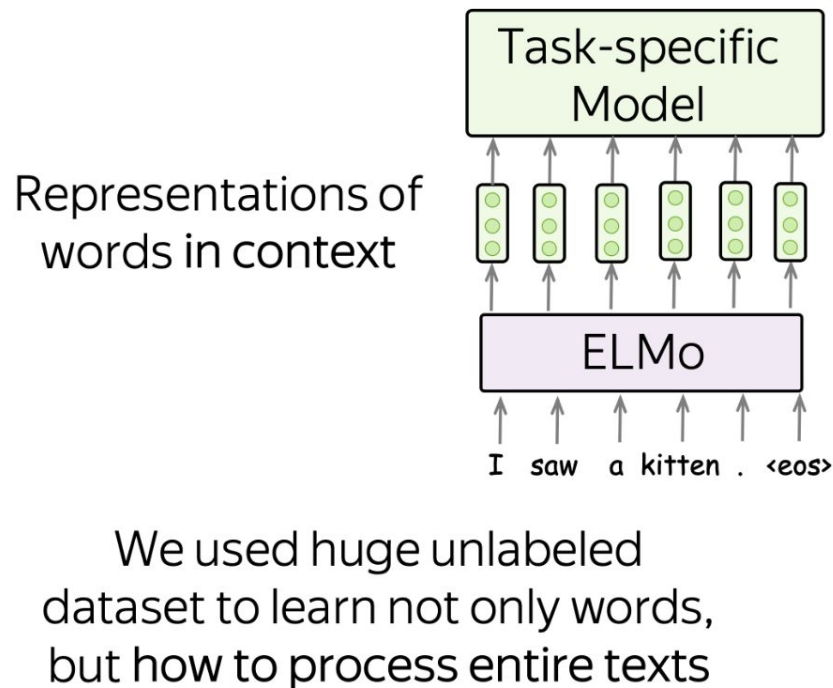


Transfer through representation of words in context

Before

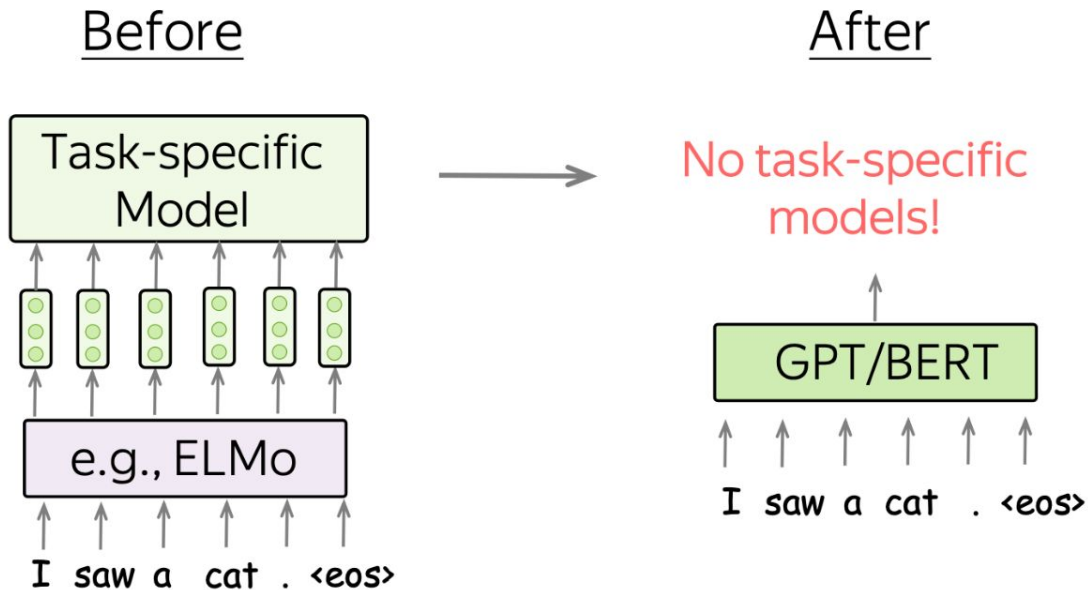


After



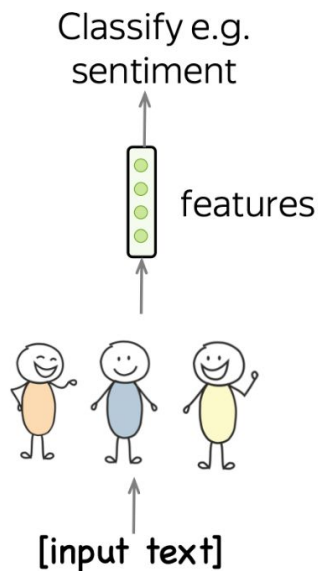
Transfer through unified models

Idea: Instead of replacing word embeddings, replace entire models.

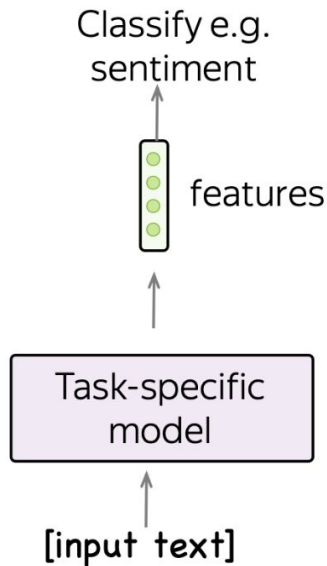


The Evolutionary Journey in NLP

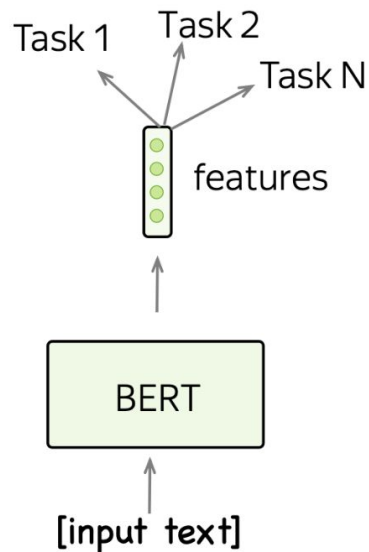
Different task –
another set of
features (and people!)



Different task –
another model



One model,
classify anything



Talk to the model

Input (prompt)

**What is the sentiment
of the next sentence?
I love this movie!**

Model output

positive

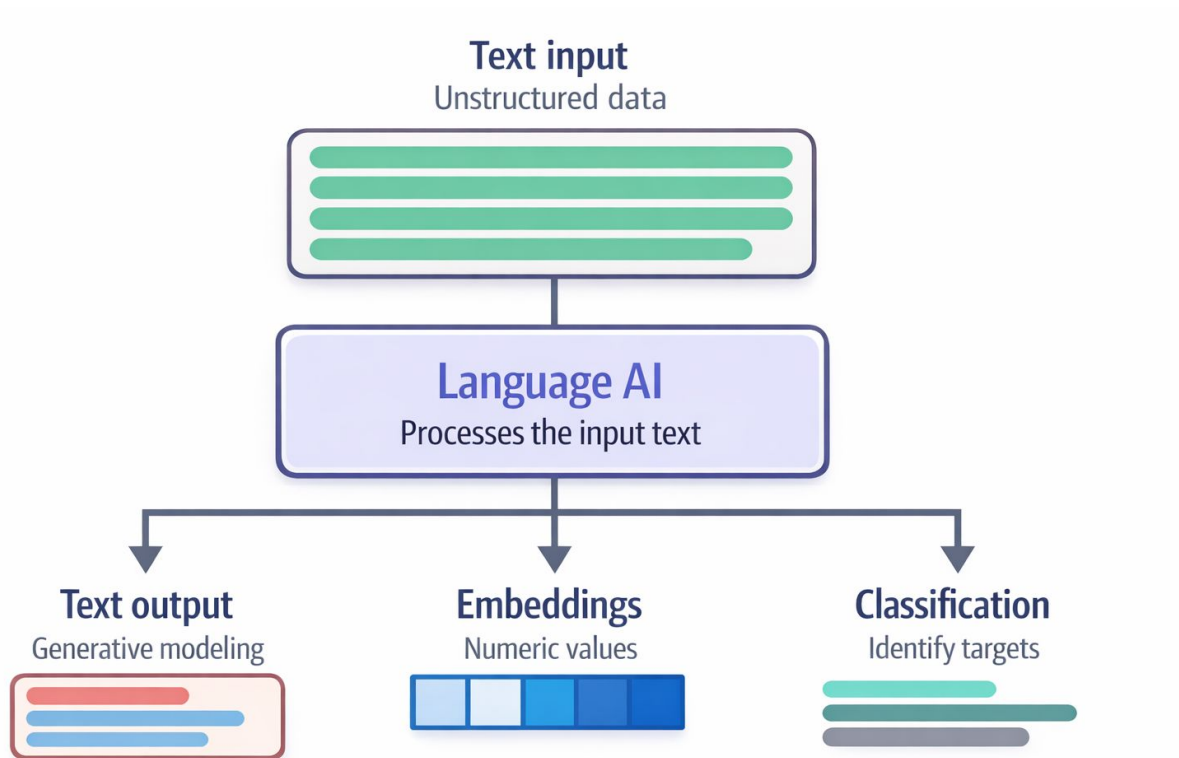
NLP Tasks



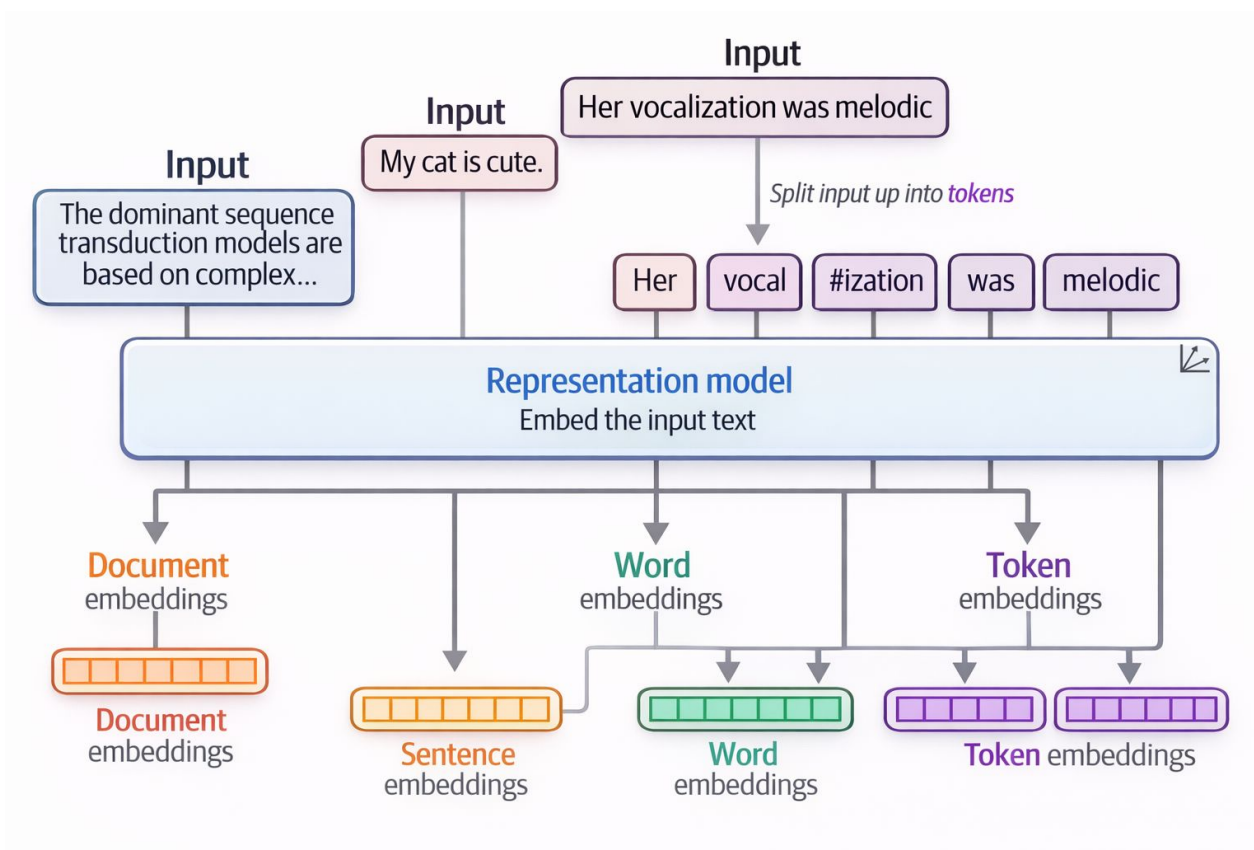
Image by NLPlanet



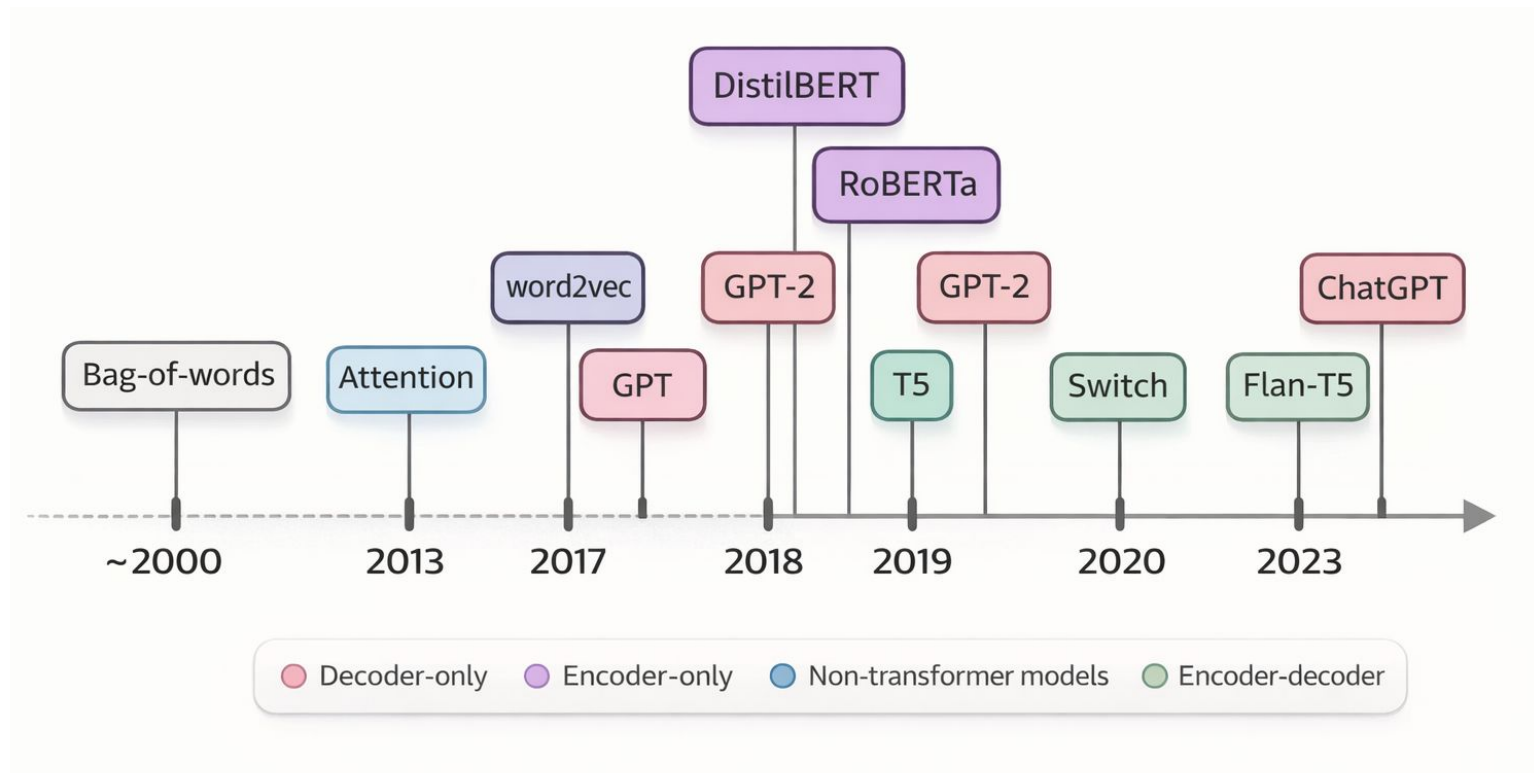
Unified model



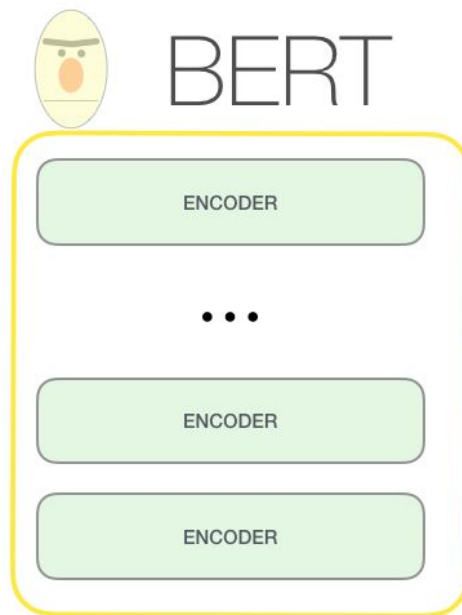
Unified model



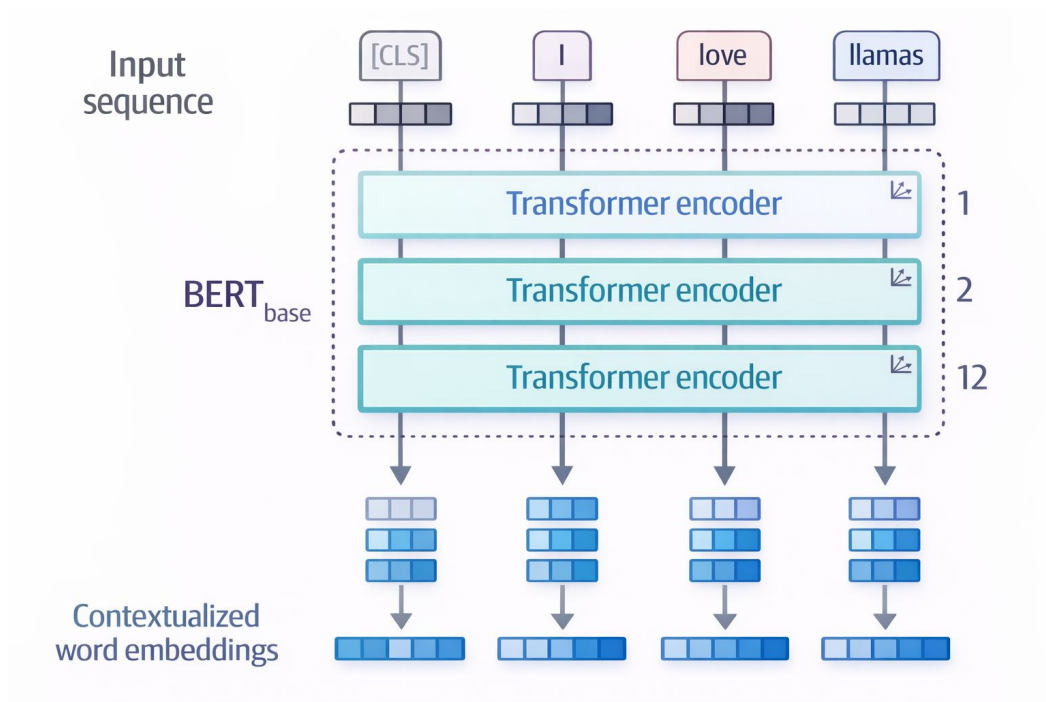
Transformers in NLP Journey



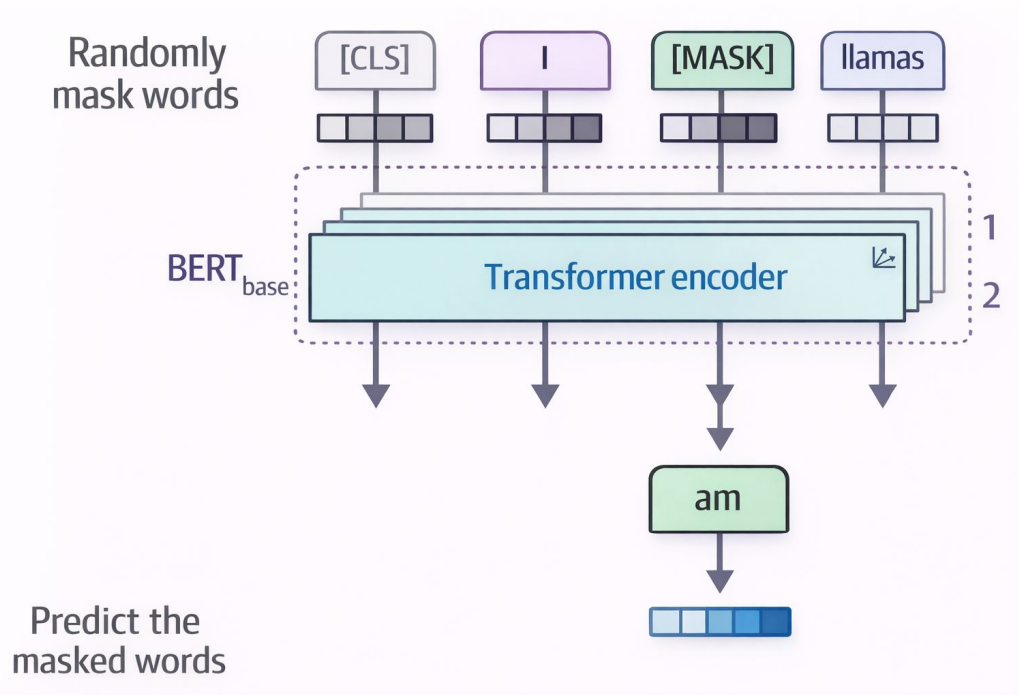
Transformers in NLP Journey



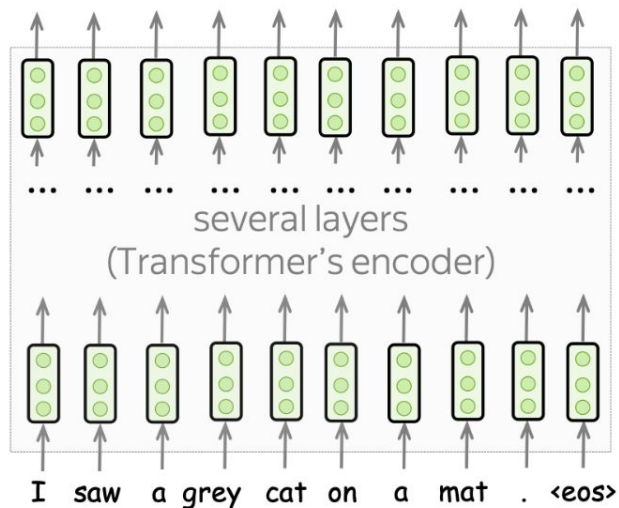
BERT: Bidirectional Encoder Representations from Transformers



BERT: Masked Language Modeling



BERT



Model architecture:

- Transformer encoder

What is special about it:

- Training objectives
 - MLM: Masked language modeling
 - NSP: Next sentence prediction
- Lots of data

BERT Input

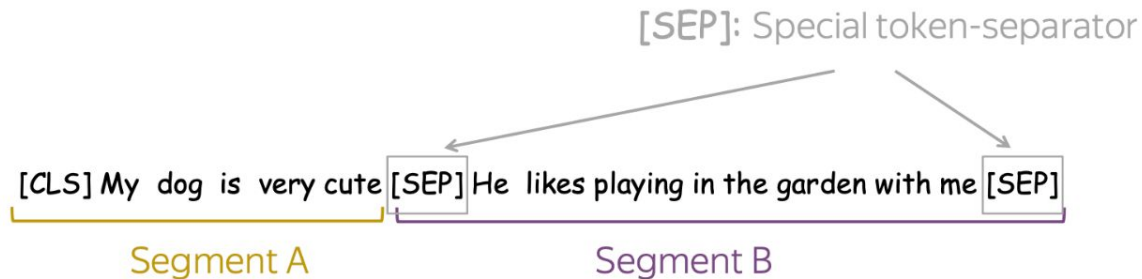
[CLS] My dog is very cute [SEP] He likes playing in the garden with me [SEP]

Segment A

Segment B

Training on pairs of sentences: either
consecutive or random (50%/50%)

BERT Input



Training on pairs of sentences: either
consecutive or random (50%/50%)

BERT Input

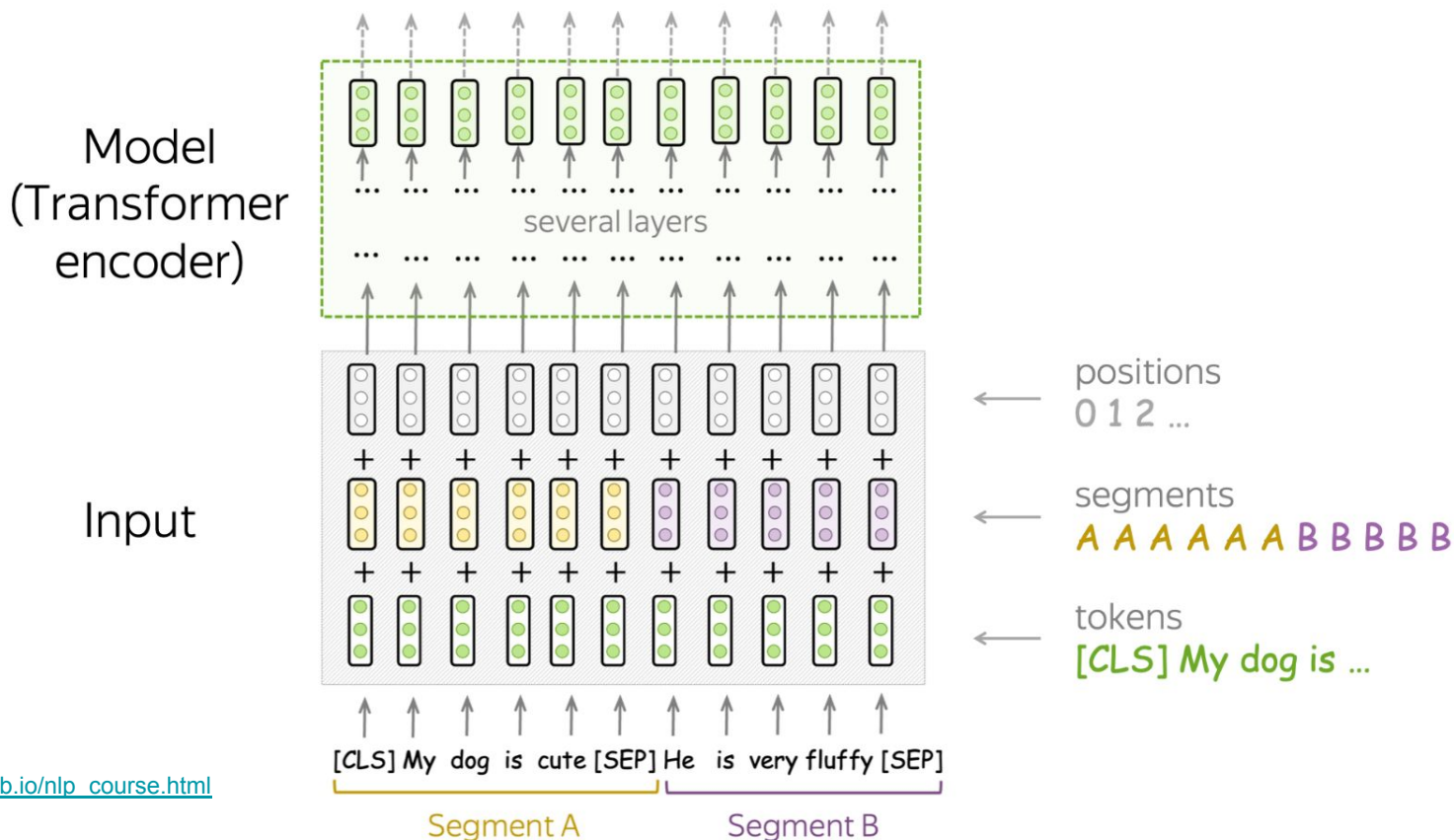
[CLS]: Special token

- Training time: predict if sentences are consecutive or not (Next Sentence Prediction /NSP objective)
- Test time: downstream tasks (e.g., classification)

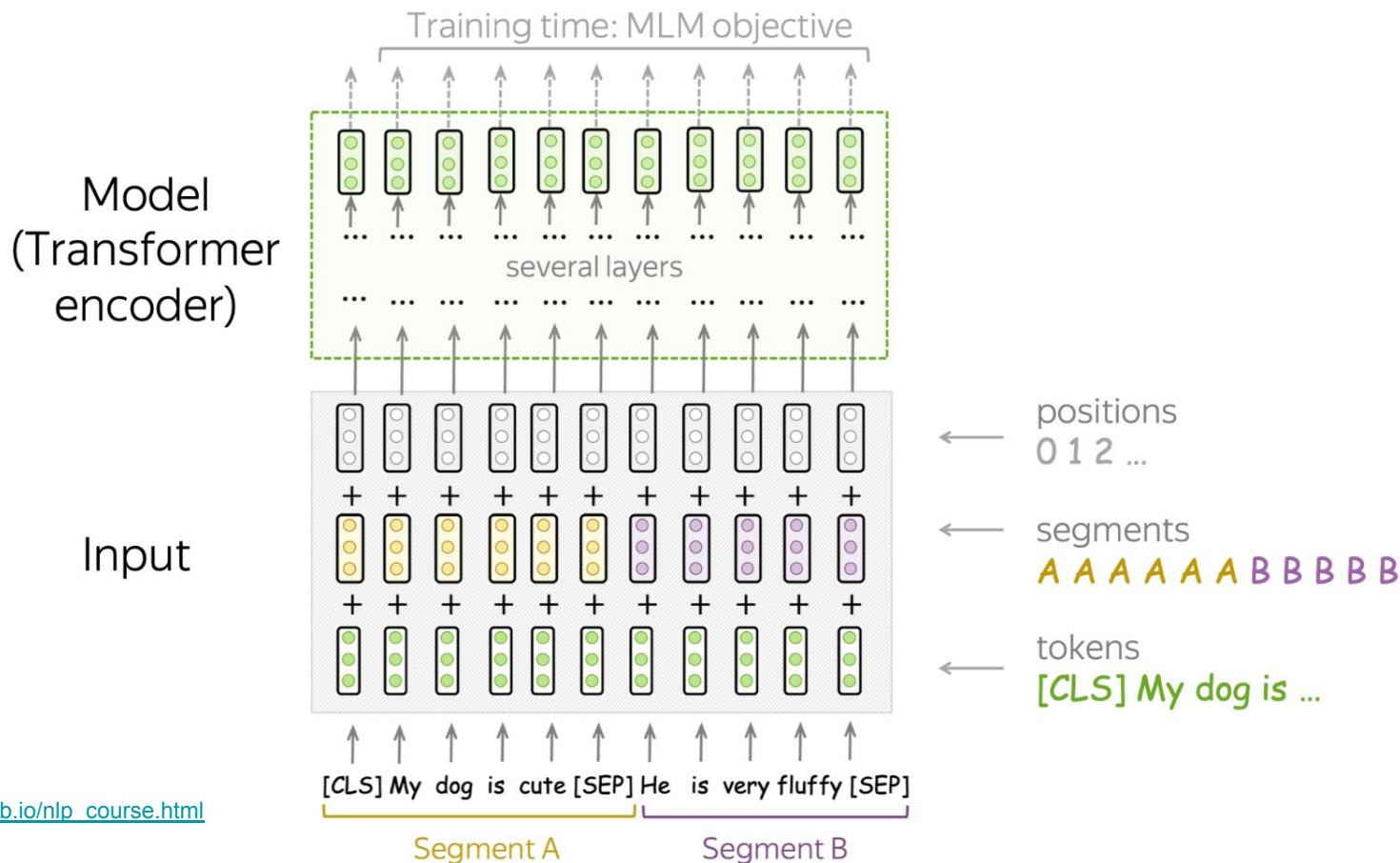


Training on pairs of sentences: either consecutive or random (50%/50%)

BERT Input



BERT Input



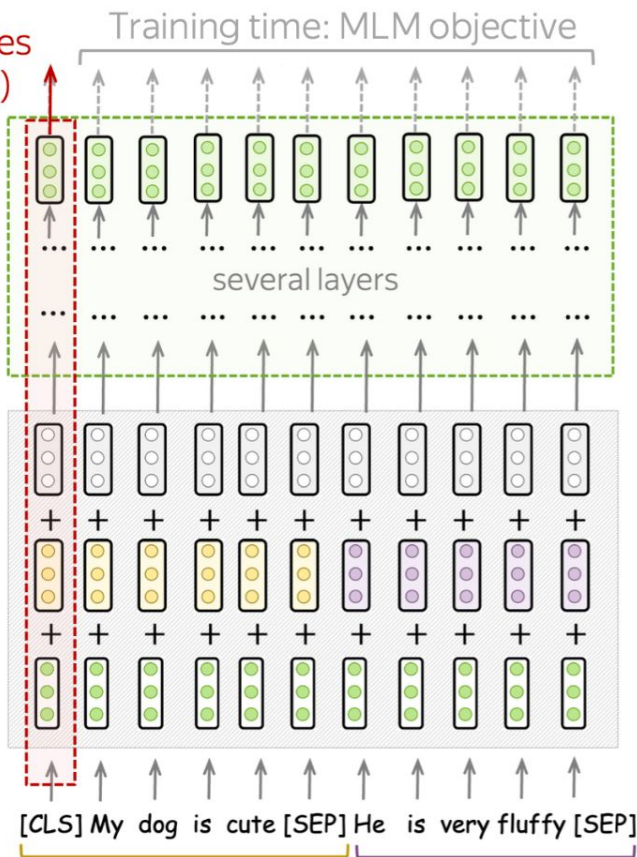
BERT: NSP objective

Training time: predict if sentences
are consecutive (NSP objective)

Test time: classification

Model
(Transformer
encoder)

Input



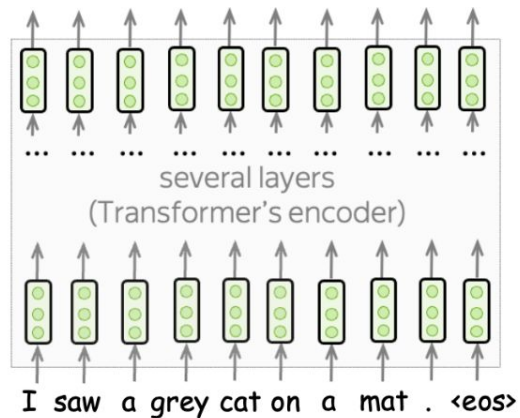
← positions
0 1 2 ...

← segments
A A A A A A B B B B B

← tokens
[CLS] My dog is ...

BERT: MLM objective

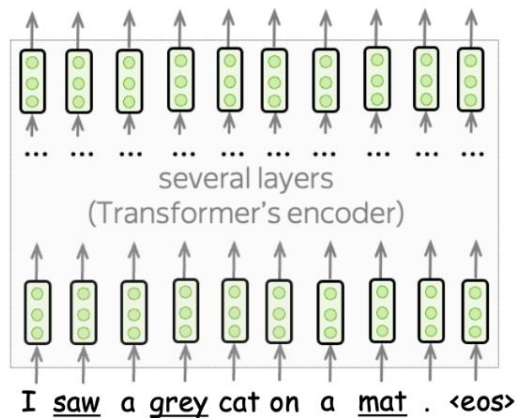
At each training step:



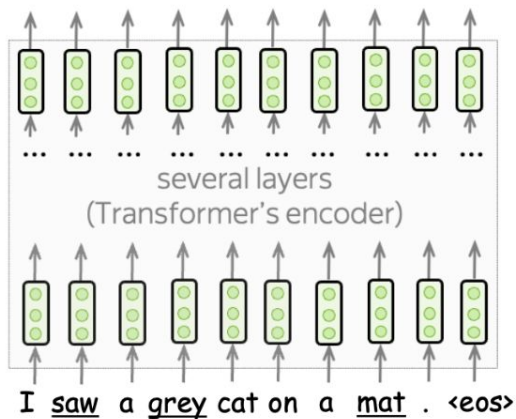
BERT: MLM objective

At each training step:

- pick randomly 15% of tokens

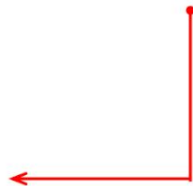


BERT: MLM objective

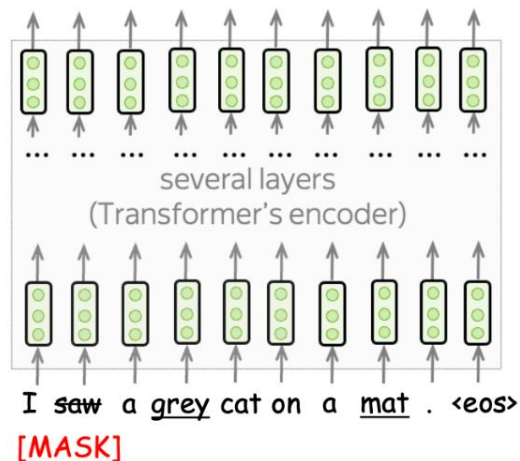


At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something



BERT: MLM objective



- [MASK],
with $p = 80\%$

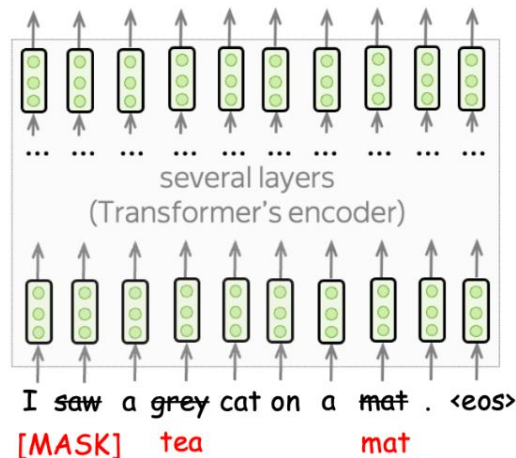
At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something

BERT: MLM objective

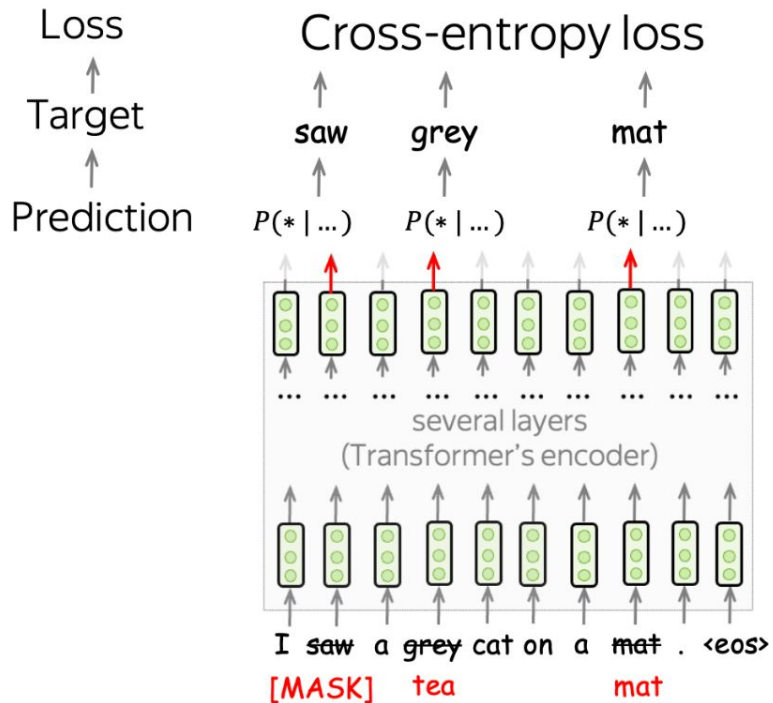
At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something



- **[MASK]**,
with $p = 80\%$
- Random token,
with $p = 10\%$
- Original token,
with $p = 10\%$

BERT: MLM objective



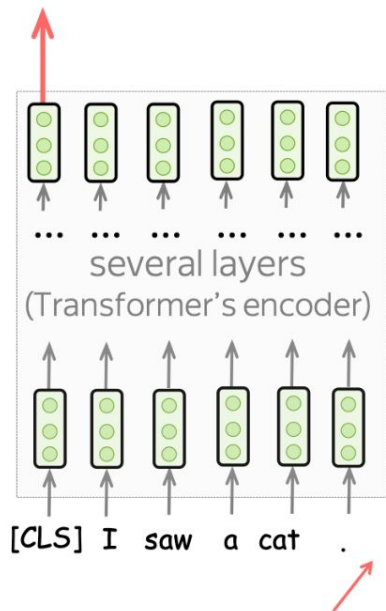
- **[MASK]**, with $p = 80\%$
- Random token, with $p = 10\%$
- Original token, with $p = 10\%$

At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something
- predict original chosen tokens

BERT finetuning: Single Sentence Classification

class label



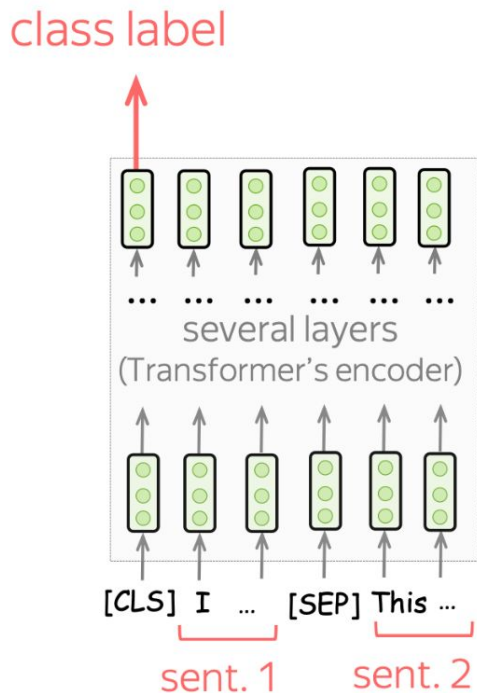
Examples of tasks:

- SST-2 – binary sentiment classification (we saw it in the text classification lecture)
- CoLA (Corpus of Linguistic Acceptability) – say whether a sentence is linguistically acceptable

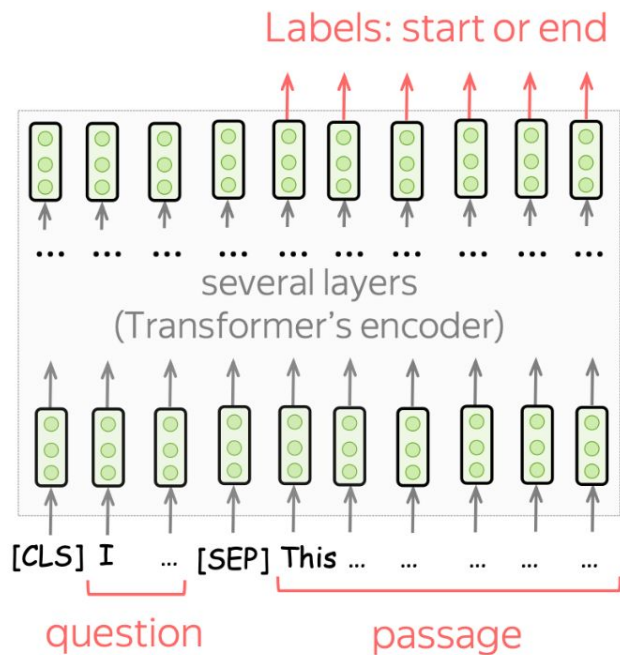
BERT finetuning: Sentence Pair Classification

Examples of tasks:

- MLNI – entailment classification. Given a pair of sentences, say if the second is an **entailment**, **contradiction** or **neutral**
- QQP (Quora Question Pairs) – given two questions say if they are semantically equivalent
- STS-B – given two sentences return a similarity score from 1 to 5



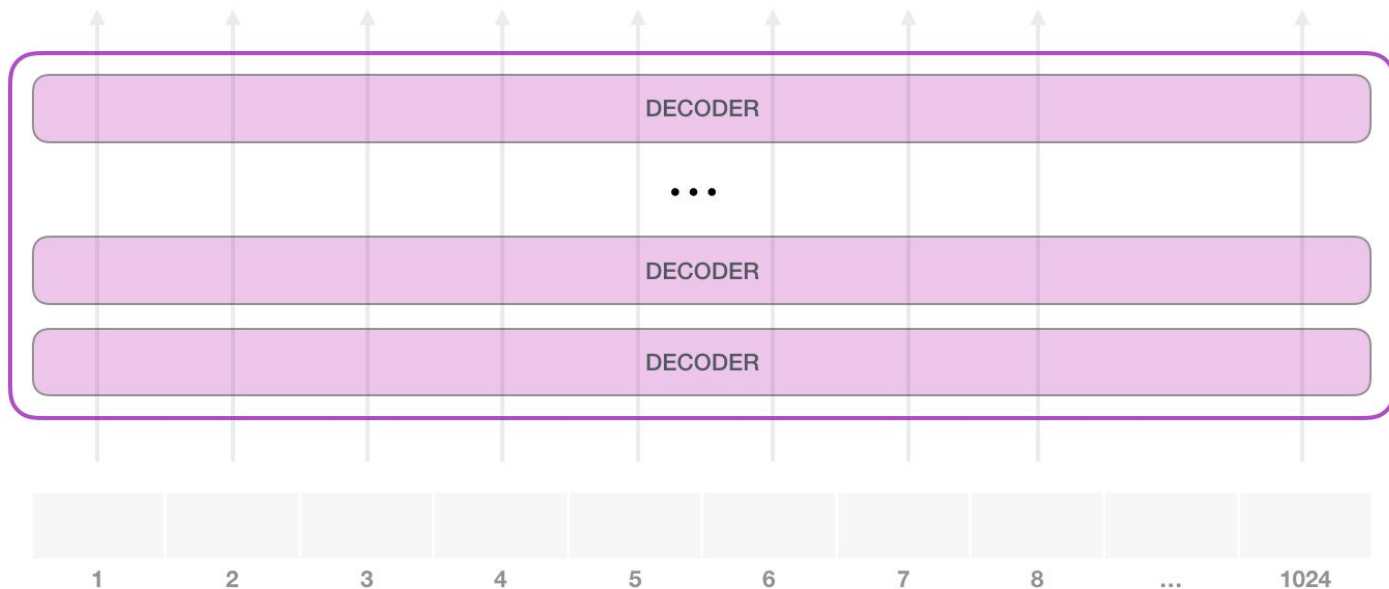
BERT finetuning: Question Answering



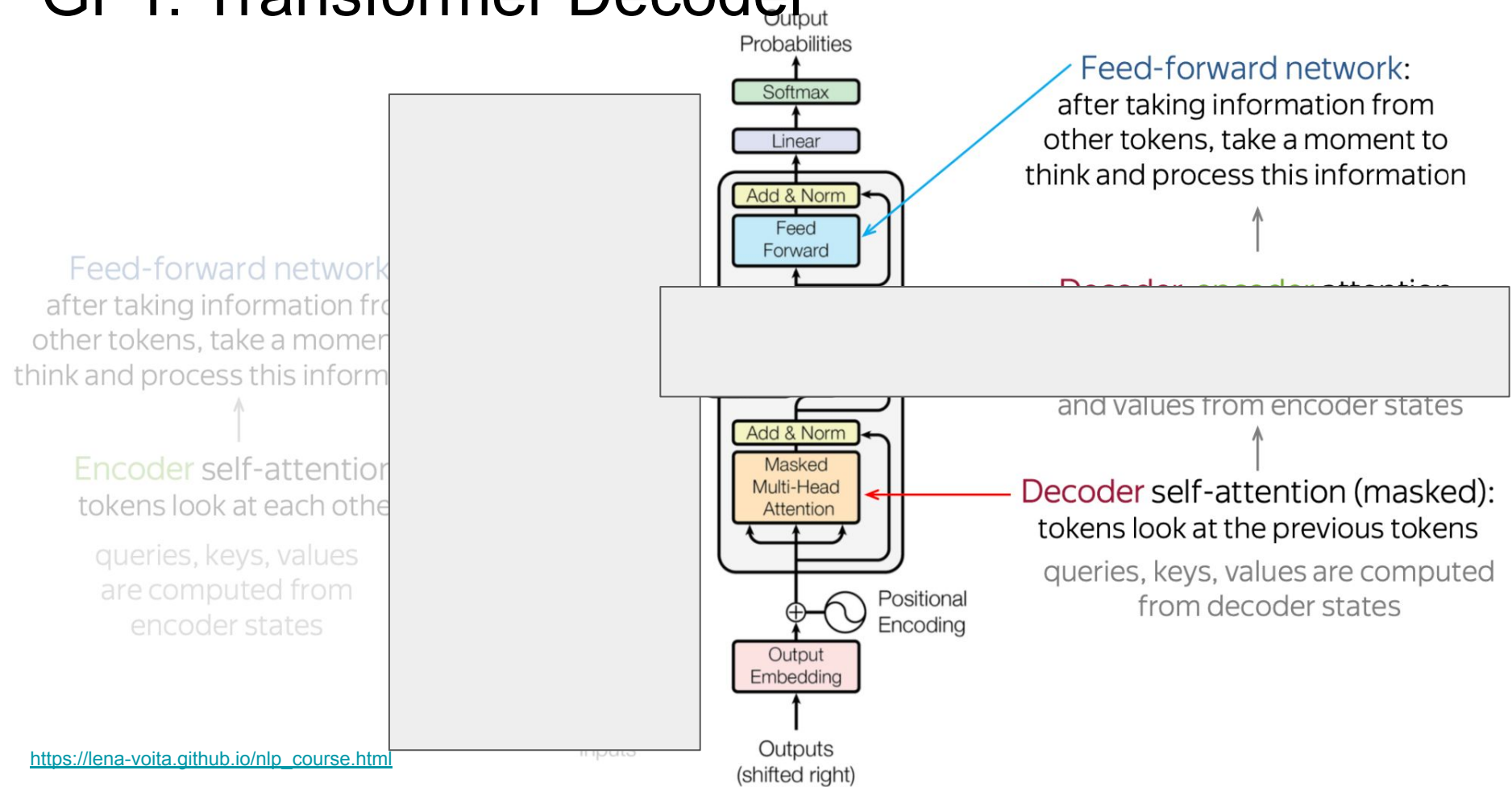
Examples of tasks:

- SQUAD – dataset with pairs of question-passage; the passage contains the answer – need to indicate where

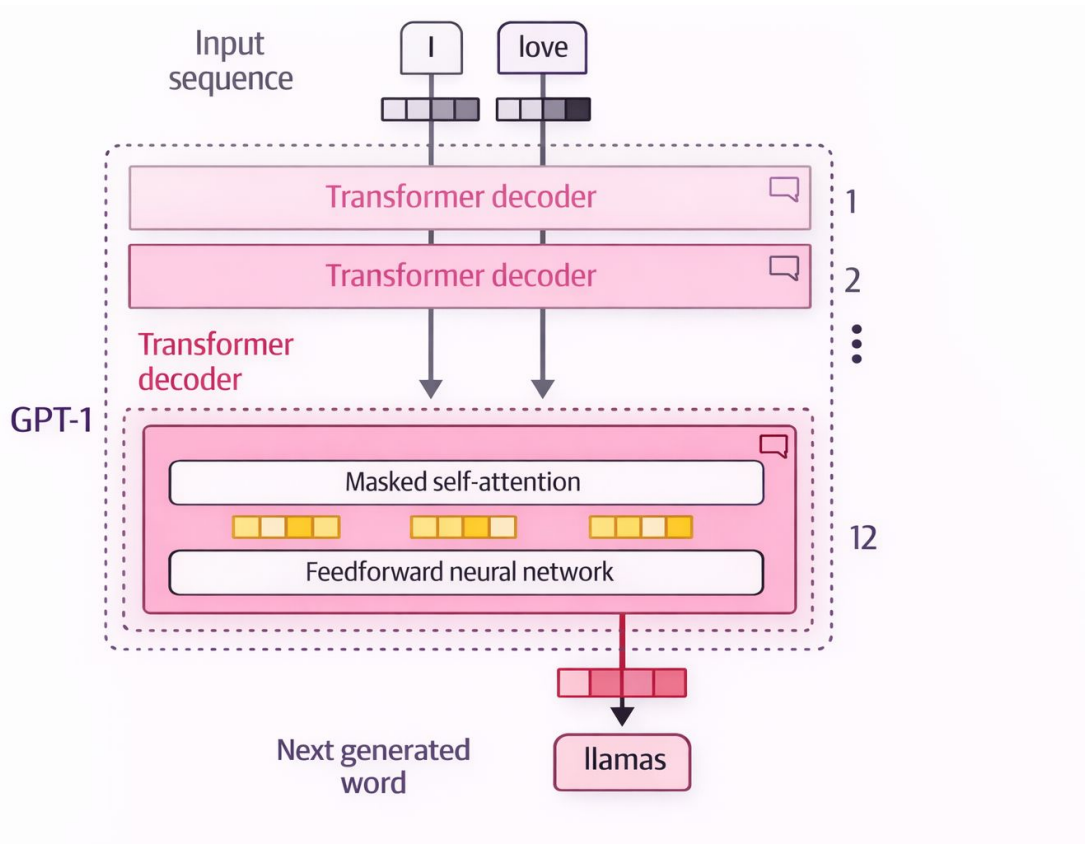
GPT: Generative Pre-trained Transformer



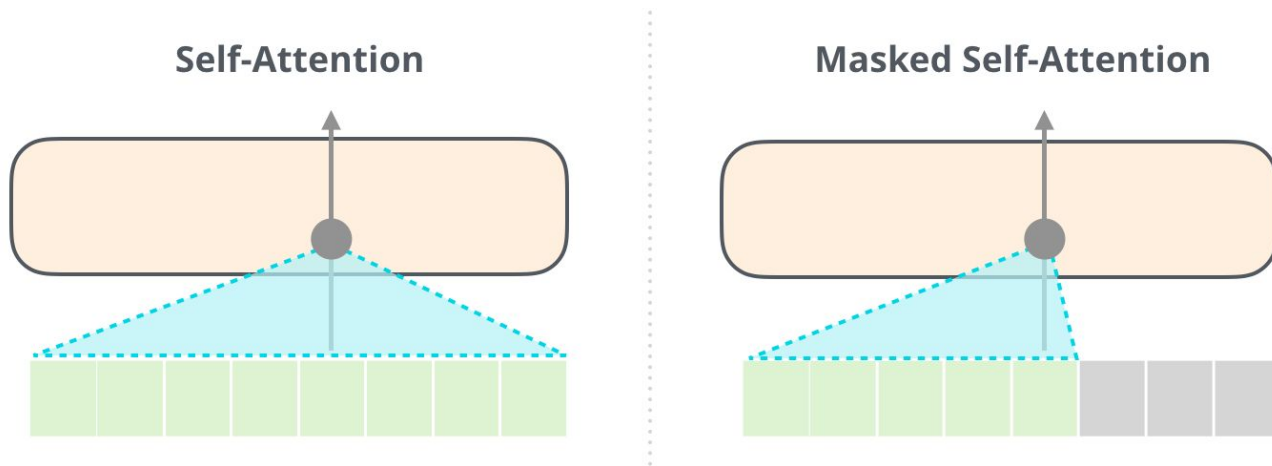
GPT: Transformer Decoder



GPT: Generative Pre-trained Transformer



GPT: Generative Pre-trained Transformer

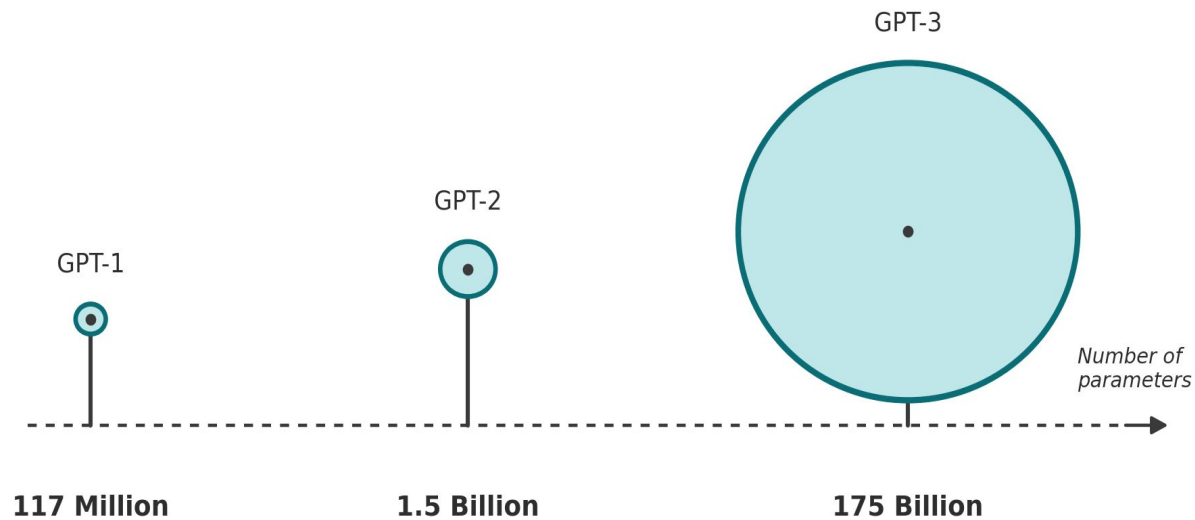


GPT: Generative Pre-trained Transformer

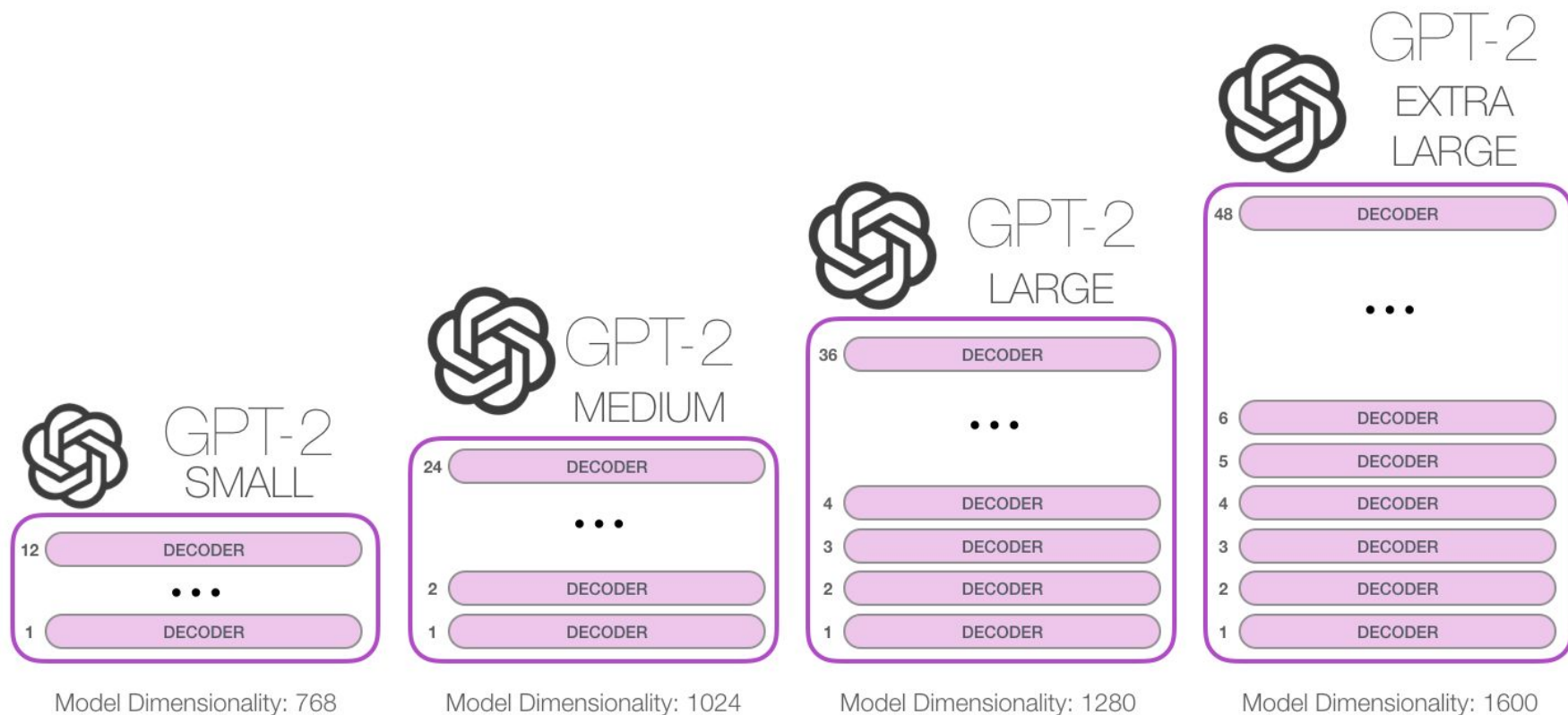
	<u>GPT-1</u> (2018)	<u>GPT-2</u> (2019)	<u>GPT-3</u> (2020)
Number of parameters	117 million	1.5 billion	175 billion
Training data	5 GB	40 GB	45 TB (i.e., 45 000 GB)

The entirety of English
Wikipedia constitutes just
0.6% of GPT-3 training data

GPT models size comparison



GPT models size comparison



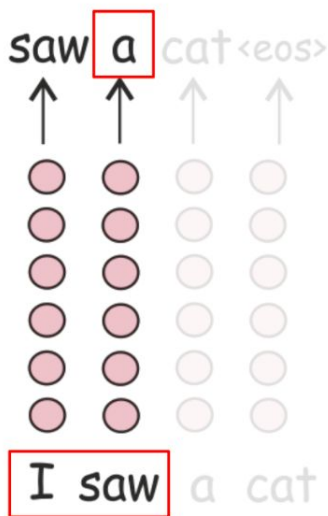
Objective: LM vs MLM

Language Modeling

Target: next token

Prediction: $P(* | \text{I saw})$

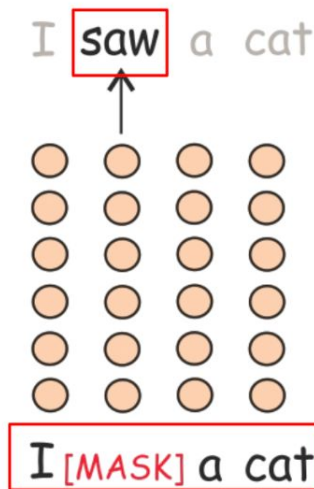
left-to-right, does
not see future



Masked Language Modeling

Target: current token (the true one)

Prediction: $P(* | \text{I [MASK] a cat})$



sees the whole text, but
something is corrupted

11 Friends of BERT



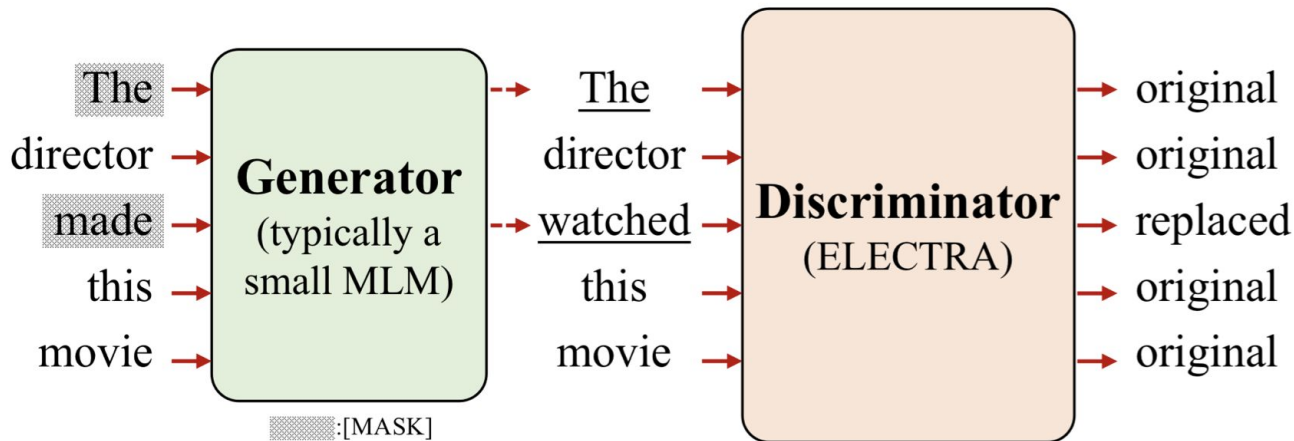
RoBERTa - doing BERT properly

- Dynamic masking: new random mask each epoch (see table)
- Tune parameters ;) the original BERT didn't train to convergence!
- Play with inputs and losses: NSP is not necessary!
- Feed it with more data!

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	94.6/89.4	90.2	96.4
BERT _{LARGE}						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7

ELECTRA

Two models generator and discriminator (see figure below)



Results: faster / cheaper training, final model $\approx$ RoBERTa

ELECTRA

Results: faster / cheaper training, final model $\approx$ RoBERTa

Model	Train / Infer FLOPs	Speedup	Params	Train Time + Hardware	GLUE
ELMo	3.3e18 / 2.6e10	19x / 1.2x	96M	14d on 3 GTX 1080 GPUs	71.2
GPT	4.0e19 / 3.0e10	1.6x / 0.97x	117M	25d on 8 P6000 GPUs	78.8
BERT-Small	1.4e18 / 3.7e9	45x / 8x	14M	4d on 1 V100 GPU	75.1
BERT-Base	6.4e19 / 2.9e10	1x / 1x	110M	4d on 16 TPUv3s	82.2
ELECTRA-Small	1.4e18 / 3.7e9	45x / 8x	14M	4d on 1 V100 GPU	79.9
50% trained	7.1e17 / 3.7e9	90x / 8x	14M	2d on 1 V100 GPU	79.0
25% trained	3.6e17 / 3.7e9	181x / 8x	14M	1d on 1 V100 GPU	77.7
12.5% trained	1.8e17 / 3.7e9	361x / 8x	14M	12h on 1 V100 GPU	76.0
6.25% trained	8.9e16 / 3.7e9	722x / 8x	14M	6h on 1 V100 GPU	74.1
ELECTRA-Base	6.4e19 / 2.9e10	1x / 1x	110M	4d on 16 TPUv3s	85.1

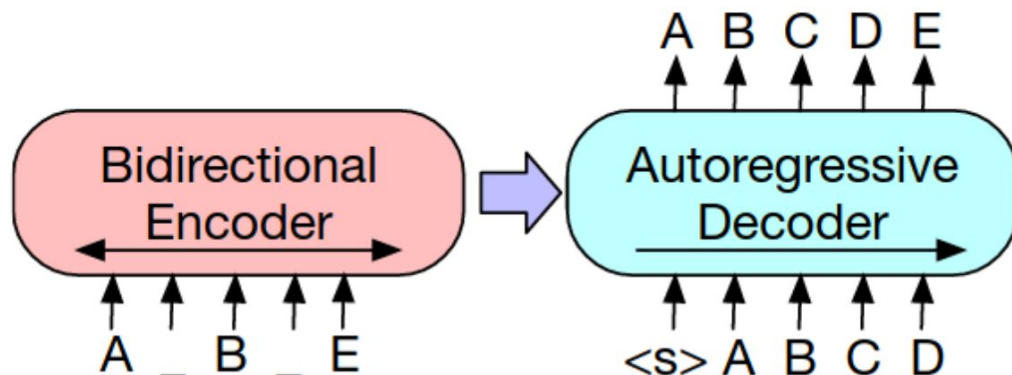
BART

BERT: full attention, but outputs are predicted independently

GPT: structured prediction, but past tokens cannot into future

BART: full attention (encoder) and structured prediction (decoder)

....and a ton of small hacks, but the main difference is seq2seq



BART

Model	SQuAD 1.1 F1	MNLI Acc	ELI5 PPL	XSum PPL	ConvAI2 PPL	CNN/DM PPL
BERT Base (Devlin et al., 2019)	88.5	84.3	-	-	-	-
Masked Language Model	90.0	83.5	24.77	7.87	12.59	7.06
Masked Seq2seq Language Model	87.0	82.1	23.40	6.80	11.43	6.19
Permutated Language Model	76.7	80.1	21.40	7.00	11.51	6.56
Multitask Masked Language Model	89.1	83.7	24.03	7.69	12.23	6.96
	89.2	82.4	23.73	7.50	12.39	6.74
BART Base						
w/ Token Masking	90.4	84.1	25.05	7.08	11.73	6.10
w/ Token Deletion	90.4	84.1	24.61	6.90	11.46	5.87
w/ Text Infilling	90.8	84.0	24.26	6.61	11.05	5.83
w/ Document Rotation	77.2	75.3	53.69	17.14	19.87	10.59
w/ Sentence Shuffling	85.4	81.5	41.87	10.93	16.67	7.89
w/ Text Infilling + Sentence Shuffling	90.8	83.8	24.17	6.62	11.12	5.41

T5 – combine best practices

- Encoder-model (like BART)
- Model & training hacks (relative pos.emb, modified objective)
- Large model, huge data

